
Point Cloud Object Segmentation Using PointNet and PointNet++

Romina Bahrami

Department of Electrical and Computer Engineering
University of Toronto
romina.bahrami@mail.utoronto.ca

Nilofer Hyder

Department of Electrical and Computer Engineering
University of Toronto
nilofer.hyder@mail.utoronto.ca

Nagham Sabbour

Department of Electrical and Computer Engineering
University of Toronto
n.sabbour@mail.utoronto.ca

Abstract

This project investigates 3D point cloud object part segmentation using PointNet and PointNet++ on the ShapeNet Part dataset. We implement a PointNet++ model in PyTorch, including hierarchical sampling, local feature learning, and feature propagation. A preprocessing pipeline is developed to normalize and resample point clouds. Performance is evaluated using accuracy and mean Intersection over Union (mIoU). Our model achieves 89% accuracy and 0.74 instance mIoU, outperforming PointNet (87%) but remaining below the PointNet++ baseline (92%), highlighting the impact of architectural capacity on segmentation performance.

Attestation of Teamwork

Romina Bahrami: Designed and implemented the PointNet++ model; conducted training, testing, and model optimization.

Nilofer Hyder: Reviewed literature and formulated the problem; trained the PointNet++ baseline; implemented point cloud visualization with color-coded part labels.

Nagham Sabbour: Handled data acquisition and preprocessing for ShapeNet Part; trained the PointNet baseline; implemented evaluation metrics and generated plots.

1 Introduction

Point cloud segmentation is fundamental in applications such as robotics and autonomous systems, enabling direct reasoning in 3D space. Unlike grid-based methods, PointNet and PointNet++ process raw point sets without voxelization Qi et al. [2016, 2017].

In this work, we implement and evaluate PointNet++ for object part segmentation on the ShapeNet Part dataset Yi et al. [2016], Chang et al. [2015], and compare it with PointNet and established baselines.

2 Preliminaries and Problem Formulation

A point cloud is an unordered set of 3D points, making it incompatible with standard convolutional networks Qi et al. [2016]. PointNet addresses this by learning permutation-invariant global features, but lacks local structure modeling. PointNet++ extends this with hierarchical grouping to capture both local and global context Qi et al. [2017].

The goal of this project is to implement PointNet++ from scratch for object part segmentation on the ShapeNet Part dataset and evaluate its performance against PointNet and PointNet++ baselines. We focus on the impact of hierarchical feature extraction and multi-scale grouping, using accuracy and mean Intersection over Union (mIoU) as evaluation metrics.

3 Design

Our PointNet++ implementation takes batched point clouds $P = \{x_i\}_{i=1}^N$, $x_i \in \mathbb{R}^3$, using normalized (x, y, z) coordinates as input features. The model learns hierarchical representations via **set abstraction (SA)** layers, followed by **feature propagation (FP)** layers for part segmentation Qi et al. [2017].

3.1 Set Abstraction

Two consecutive SA layers perform sampling, grouping, and local feature learning. Centroids are selected using farthest point sampling, and local neighborhoods are formed via ball queries. Each neighborhood is processed by a shared MLP with max pooling (PointNet) to produce invariant local features.

To handle varying densities, we use **multi-scale grouping (MSG)**, where features from multiple radii are extracted and concatenated. We also implement **multi-resolution grouping (MRG)**, which combines features from earlier layers with raw-point features to adapt to local density.

3.2 Feature Propagation

FP layers upsample features back to the original resolution using inverse-distance weighted interpolation:

$$f(x) = \frac{\sum_i w_i(x) f_i}{\sum_i w_i(x)}, \quad w_i(x) = \frac{1}{\|x - x_i\|}$$

We use two FP blocks to progressively fuse features from abstraction layers with the original points.

3.3 Final Prediction

A shared MLP followed by a linear layer predicts per-point part labels.

The implemented structure is demonstrated in figure 1.

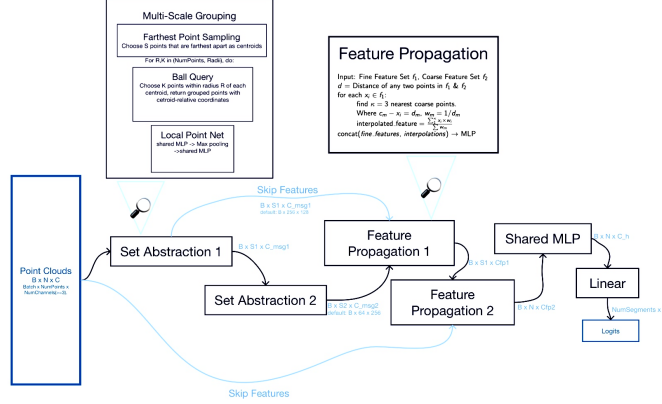


Figure 1: PointNet++ architecture with Multiscale Grouping for part segmentation

4 Methodology

The implementation is available in the project repository Romina Bahrami [2026]. The main components are summarized below.

In addition, notebooks/pointnetpp_partseg_demo_notebook.ipynb provides a demo of how the full prediction pipeline works.

4.1 Data Preprocessing

We use the ShapeNet Part dataset Yi et al. [2016], Chang et al. [2015], Google Drive source [2026] with official splits and category mappings Qi et al. [2016, 2017].

All point clouds are normalized by centering at the origin and scaling to the unit sphere. Each sample is resampled to a fixed size of 1024 points: sampling without replacement for larger point sets and with replacement for smaller ones, while preserving label alignment. As shown in Figure 2, the dataset is imbalanced across categories, and the number of part labels varies significantly between classes.

The processed data is implemented as a PyTorch Dataset containing point coordinates, class labels, and segmentation labels. The corresponding modules are src/preprocessing.py and src/dataset.py.

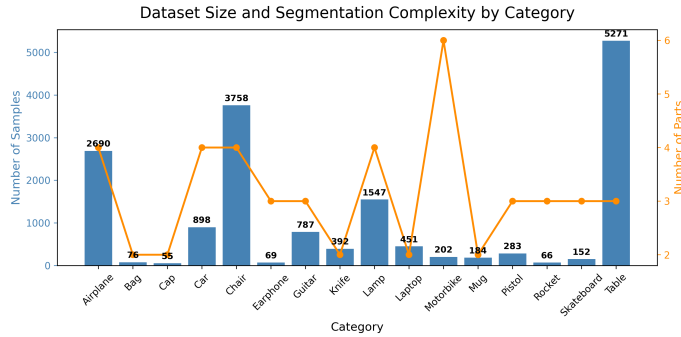


Figure 2: Distribution of samples per category (bars) and number of part labels per category (line) in the ShapeNet Part dataset.

4.2 Model Implementation

Our model, in src/pointnetpp/, follows PointNet++ architecture in figure 1. Set Abstraction (SA) layers perform farthest-point sampling, ball-query grouping, and shared MLP-based feature extraction,

with multi-scale grouping (MSG) and multi-resolution grouping (MRG). Feature Propagation (FP) layers upsample features via inverse-distance interpolation with skip connections, followed by MLP refinement. A final shared MLP predicts per-point labels.

The model is implemented in PyTorch and trained with test-time voting. Due to computational constraints, it uses 2 SA and 2 FP layers, compared to 3 in the baselines Yan [2019].

In addition to part segmentation, our implementation supports object classification using both MSG and MRG. However, the results of the multi-resolution grouping and object classification are not explained here since they are beyond the scope of this report.

4.3 Evaluation

Evaluation is performed using overall accuracy, class-average accuracy, class-average mIoU, and instance-average mIoU. Predictions are obtained by taking the argmax over model outputs.

Overall accuracy measures the fraction of correctly classified points. Class-average accuracy computes accuracy per category to reduce bias toward larger classes.

For segmentation quality, we use mean Intersection over Union (mIoU), computed over valid parts for each shape. Instance-average mIoU averages across all shapes, while class-average mIoU averages across categories for a balanced evaluation.

To improve robustness, we apply test-time voting by averaging predictions over multiple perturbed inputs. The evaluation pipeline is implemented in `src/Utils/partseg_metrics.py`.

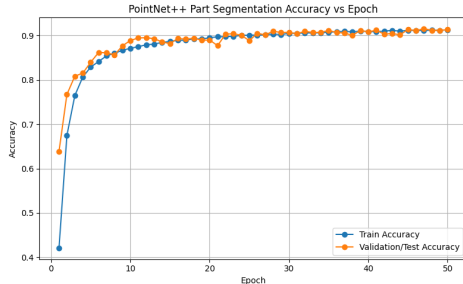
5 Numerical Experiments

The model was trained for 50 epochs (best checkpoint at epoch 47) using hyperparameters aligned with the baseline where possible. Training was performed on an M1 MacBook (16GB RAM). Learning curves are shown in Figure 3, and quantitative results are summarized in Tables 1 and 2.

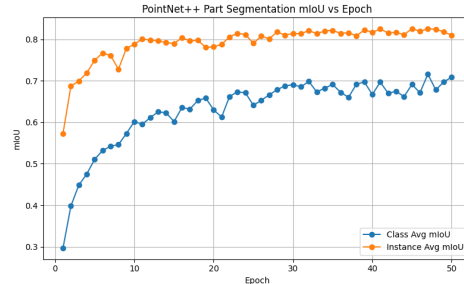
We evaluate our implementation on the ShapeNet Part dataset and compare it against PointNet and PointNet++ baselines trained using Yan [2019] (see `/pointnet_baseline` and `/pointnet2_baseline`). While our architecture follows the PointNet++ design, it uses 2 set abstraction (SA) and 2 feature propagation (FP) layers instead of 3, due to computational constraints.

On the test set, using 3-vote evaluation with Gaussian jittering ($\sigma = 0.01$), our model achieves 0.893 ± 0.004 point accuracy, 0.694 class mIoU, and 0.785 instance mIoU. This improves over the PointNet baseline, but remains below the PointNet++ MSG benchmark, which is expected given the reduced depth of our model.

Qualitatively, the model performs well on structured categories such as Chair, Guitar, and Laptop, but struggles on classes with fewer samples or higher structural variability, such as Motorbike and Rocket. Additional visualizations are provided in the repository, under `/visuals`, and instructions on how to run the experiments and reproduce the results can be found in `/README.md`.



(a) Training and Validation Loss over Epochs



(b) mIoU over Epochs

Figure 3: Learning curves of the model. Left: training and validation loss over epochs. Right: mean Intersection-over-Union (mIoU) over epochs.

Table 1: Comparison of part segmentation performance on the ShapeNet Part test split.

Model	Point Accuracy	Class mIoU	Instance mIoU
PointNet baseline	0.875	0.661	0.735
Our custom PointNet++	0.893 ± 0.004	0.697	0.787
PointNet++ MSG baseline	0.920	0.740	0.813

Table 2: Selected per-class mIoU of our model.

Top Classes	mIoU	Bottom Classes	mIoU
Laptop	0.903	Rocket	0.299
Chair	0.873	Motorbike	0.319
Mug	0.871	Earphone	0.594
Guitar	0.870	Bag	0.602

6 Discussion

The results show that our implementation of PointNet++ is effective for point cloud part segmentation, achieving strong performance across most categories. In particular, high mIoU scores on structured objects such as Chair, Guitar, and Laptop indicate that the model successfully captures both global shape and local geometric features.

However, performance is lower on categories such as Motorbike and Rocket. These classes tend to have fewer training samples and more complex or less consistent structures, making them more difficult to segment accurately. This highlights the sensitivity of the model to dataset imbalance and variability in object geometry.

Compared to the PointNet baseline, our model demonstrates clear improvements, confirming the benefit of hierarchical feature learning. However, it does not fully match the performance of the PointNet++ baseline. This gap is likely due to the reduced model depth in our implementation (2 SA and 2 FP layers instead of 3), which limits its ability to capture higher-level features.

Overall, the results suggest that while our implementation captures the core advantages of PointNet++, model capacity and dataset characteristics play a significant role in achieving state-of-the-art performance.

7 Conclusions

To conclude, our implementation of PointNet++ achieved a final accuracy of approximately 89%, outperforming the PointNet baseline while remaining below the PointNet++ baseline. This performance gap is mainly due to reduced model capacity, as our implementation uses fewer abstraction and propagation layers while keeping the overall design consistent with the original architecture.

Future improvements could focus on applying data augmentation to underrepresented categories to improve generalization. Additionally, incorporating object category information as input may further enhance segmentation performance. Another potential direction is to improve convergence speed by introducing batch normalization in the feature propagation (upsampling) layers.

References

- Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015.
- Google Drive source. Shapenet part annotation benchmark dataset. <https://drive.usercontent.google.com/download?id=1W3SEE-dY1sxv1ECc0wWSDYemwHEUbjIS&authuser=0>, 2026. Direct download link used for dataset acquisition.
- Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. *arXiv preprint arXiv:1612.00593*, 2016.
- Charles R Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *arXiv preprint arXiv:1706.02413*, 2017.
- Nagham Sabbour Romina Bahrami, Nilofer Hyder. 3d point cloud segmentation. <https://github.com/rominaba/3d-point-cloud-segmentation>, 2026.
- Xu Yan. Pointnet/pointnet++ pytorch. https://github.com/yanx27/Pointnet_Pointnet2_pytorch, 2019.
- Li Yi, Vladimir G. Kim, Duygu Ceylan, I-Chao Shen, Mengyan Yan, Hao Su, Chun-Yi Lu, Qixing Huang, Alla Sheffer, and Leonidas Guibas. A scalable active framework for region annotation in 3d shape collections. In *ACM Transactions on Graphics (SIGGRAPH Asia)*, 2016.

Use of AI

AI tools were used in a limited and supportive role throughout this project. During development of the PointNet++ implementation, AI was mainly used to help debug specific PyTorch issues (such as checkpoint loading mismatches and tensor shape errors), clarify certain implementation details from the original papers (e.g., set abstraction and feature propagation behavior), and assist with code adjustments. It was also used to improve the clarity and conciseness of written sections of the report.

Contest for Information Sharing

Consent for Sharing Project Materials

- All group members consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

- All group members consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Group Identification

- Group number: 31
- Names of group members: Romina Bahrami, Nilofer Hyder, Nagham Sabbour
- Signature of of group members: *Romina Bahrami, Nilofer Hyder, Nagham Sabbour*
- Date: April 10, 2026